

Evaluation of the Architecture Alternatives for Real-time Intrusion Detection Systems for Connected Vehicles

Mubark Jedh *, Jian Kai Lee*, Lotfi ben Othmane*

* Iowa State University, Ames, IA USA

Abstract—Attackers demonstrated the use of remote access to the in-vehicle network of connected vehicles to launch cyber-attacks and remotely take control of these vehicles. Machine-learning-based Intrusion Detection Systems (IDSs) techniques have been proposed for the detection of such attacks. The evaluation of some of these IDS demonstrated their efficacy in terms of accuracy in detecting message injections but was performed offline, which limits the confidence in their use for real-time protection scenarios. This paper evaluates four architecture designs for real-time IDS for connected vehicles using Controller Area Network (CAN) datasets collected from a moving vehicle under malicious speed reading message injections. The evaluation shows that a real-time IDS for a connected vehicle designed as two processes, a process for CAN Bus monitoring and another one for anomaly detection engine is reliable (no loss of messages) and could be used for real-time resilience mechanisms as a response to cyber-attacks.

I. INTRODUCTION

Modern Automobile contains 20 to 80 Electronic Control Units (ECUs) that control the functionalities of the vehicle, including engine, power steering, brake, air-conditioning, radio, and seats, lane departure alert, and emergency braking, as depicted by Figure 1. These ECUs communicate using the Controller Area Network (CAN) bus protocol [1]. The CAN protocol was designed more than 40 years ago and has still widespread use in automotive, aerospace, and many other industries because of its low cost, error detection capability, reliability, and high-speed transmission rate. The CAN protocol does not include, however, security measures such as authentication [2].

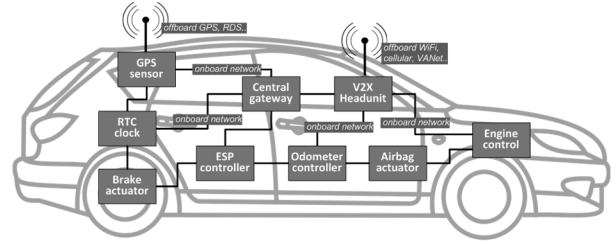


Fig. 1: Example of connected vehicle [3]

Several recent research works demonstrate that attackers can access the CAN Bus using a variety of interfaces such as TPMS, bluetooth, telematics, and OBD-II units to inject messages into the CAN Bus. Hoppe et al. [5] were the first researchers to point out the security weaknesses of the CAN Bus protocol. Their findings were later confirmed by Koscher et al. [6]. Furthermore, Checkoway et al. [7] performed a security analysis of attack surfaces, including physical, short-range and long-range wireless communication, and demonstrated the exploitation of the flaws that they have identified to fully take control of the vehicle's systems. Recently, Upstream's research team identified 367 publicly reported incidents for a decade long [4]. The analysis of these incidents shows an exponential growth of attacks, as depicted by Figure 2a. Among these attacks, 27% involved taking control of the car, as depicted by Figure 2b.

Intrusion Detection Systems (IDSs) have been proposed as an alternative to the attack prevention approach on connected vehicles. Wu et al. [8] and Young et al. [9] provide comprehensive surveys on IDS for connected and autonomous cars. In previous work [10],

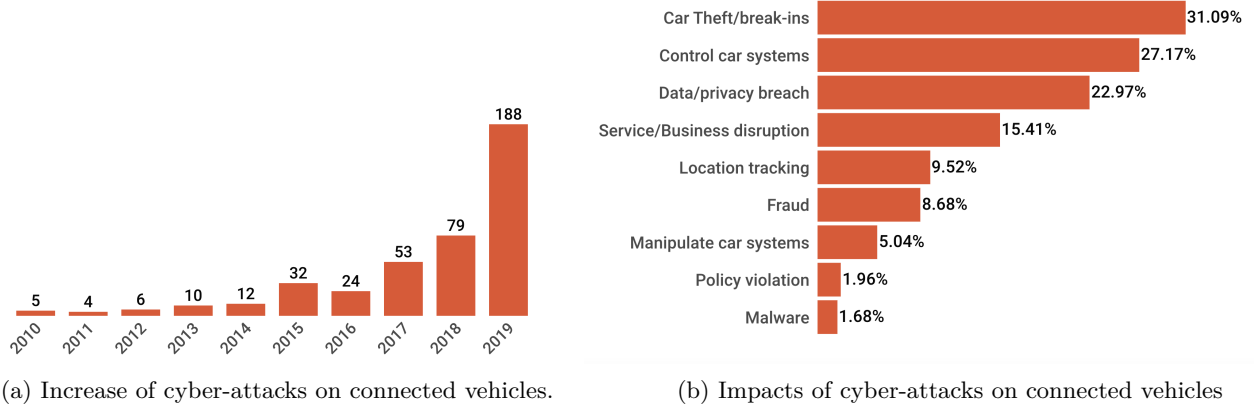


Fig. 2: Growth and distribution of cyber-attacks on connected vehicles between 2010 and 2019 [4].

TABLE I: Major requirements for IDS for connected vehicles.

ID	Requirements
1	The response time of IDS must be small enough to trigger reactive safety mechanisms, such as breaking.
2	The IDS must not lose CAN data.
3	The IDS must run on an ECUs that has limited capabilities in terms of processing speed and memory size.

[11] we developed Machine Learning (ML)-based IDSs for connected vehicles and evaluated them using CAN data extracted from a moving vehicle under malicious RPM and speed readings messages injections into the in-vehicle network of the vehicles. Most of the ML-based IDS for the connected vehicle are evaluated offline using datasets of CAN logs, including the ones that we designed, which limits the confidence in the capabilities to use them for real-time intrusion detection in vehicles.

There are three requirements for IDSs for connected vehicles, which are listed in Table I. We describe in this paper four architecture alternatives for real-time IDS for connected vehicles.¹ Then, we assess their satisfaction of the requirements by evaluating their (1) anomaly evaluation time, and (2) reliability in terms of losing CAN messages. The findings demonstrate the possibility to deploy effective ML-based IDSs for connected vehicles.

This paper is organized as follows. Section II describes

the related works, section III describes the used IDS, Section IV describes the architecture alternatives of the real-time IDS for connected vehicle, Section V describes the evaluation of the architecture alternatives, and Section VI concludes the paper.

II. RELATED WORK

Several preventive security countermeasures have been developed to defend and enhance in-vehicle network security against cyber-attacks, such as authentication protocols [12], [13], [14]. The main issue with these mechanisms is that these address only a subset of the attacks on the connected vehicles and require modification of the protocol used by the ECUs of the given vehicle, which cannot be used for aftermarket vehicles. In addition, most of the remote attacks exploit software vulnerabilities in the protection mechanisms, such as in [15], [16], [17], [18].

IDSs have been proposed as an alternative to prevention mechanisms from attacks on connected vehicles. Wu et al. [8] and Young et al. [9] provide surveys on IDS for connected and autonomous cars. These mechanisms discriminate groups of messages associated with attacks and those that are not, with acceptable accuracy and false positive [19]. Neural Networks (NN) has been the commonly used ML-based approach for designing IDSs for the CAN bus, e.g., [20], [21], [22], [23]. In previous work [10], [11] we used machine learning techniques to develop IDSs for connected vehicles including

¹We focus on CAN message injection attacks. Other attacks on connected vehicles, e.g., on V2V could be considered in the future.

Hodden Markov Model (HMM), Long Short-Term Memory (LSTM), cosine graph-similarity, and change-point detection and evaluated them using CAN data extracted from a moving vehicle under malicious RPM and speed readings messages injections into the in-vehicle network of the vehicles. The detection accuracy of cosine graph-similarity threshold reaches 97.32% of accuracy and the detection speed of 2.5 milliseconds. Unlike most ML-based studies in the literature, the technique proposed in [11] neither depend on the make or model of the car nor its proprietary information (i.e., CAN ID).

The common data sources that are used to assess the accuracy of the ML-based IDS solutions include; data from the owner devices [24], synthetic/artificial data [25], simulated data [26], and data from a stationary/-parked vehicle [27], [28], [29]. This limits the confidence in the results and threatens its validity [30]. To the best of our knowledge, our previous work [10], [11] and the works of Stachowski [23] are the only studies that used datasets collected from moving vehicles under messages injection attacks.

Valasek and Miller are among the pioneer to propose real-time IDS for connected vehicles [31]. They developed a small device that reads data from the CAN Bus through OBD-II port, learns the traffic pattern to detect anomalies, and shorts the circuit disabling all CAN messages when anomalous detected. Matsumoto et al. [32] proposed a real-time IDS that prevents authorized messages from reaching the receiver ECU. The system monitors the traffic of the CAN Bus and transmits Error Frame to override the unauthorized messages when it detects them. The technique requires, however, modification of the CAN Bus protocol.

Boddupalli and Ray assessed the requirements for real-time attack detection and mitigation in connected and autonomous vehicles and emphasized the importance of the basic safety of such a mechanism. [33], [34]. They trained a NN-based IDS and proposed an architecture that addresses the requirements for the case of collision avoidance using vehicle-to-vehicle mechanisms. The main components of the architecture are: (1) a predictor of abnormal behavior that uses the trained IDS, (2) machine learning models for computing the application decision trained from driving a vehicle in different weather conditions (windy, raining, snowing, and clear) and road types (city, suburban, and highway)

which are used to estimate the response of the module, and (3) a plausibility check module that checks the safety of using the output of the response estimator. When the system detects an anomaly, it replaces the response computed by the collision application with the output of the response estimator when the plausibility check is positive; that is, the estimated response is safe. The system triggers service degradation if it detects an anomaly and the plausibility checks of the output of the estimated response is negative.

III. INTRUSION DETECTION SYSTEM

ECUs collaborate to perform tasks such as increasing speed, break, etc., by sending messages through the CAN bus [10]. Attackers take control of a connected vehicle by injecting messages into its CAN bus. To mitigate such attacks, we have developed an ML-based IDS that captures the pattern of the sequences of the CAN messages and represent them with a directed graph, which we call *Messages-Sequence Graph (MSG)*, where the nodes represent the CAN IDs of the messages and the edges represent the sequences order of the messages, as depicted by Figure 3 [11].

To construct the MSG, we first create a dictionary of the CAN IDs exchanged in the CAN bus. Then, we label the nodes of the MSG with the CAN IDs and the edges with value "0". Next, we loop over the batch of the CAN messages of size w (e.g., 1000 successive messages) that were exchanged from time t . For each of the messages, we increase the label of the edge linking the node representing the CAN ID of the message to the node representing the CAN ID of the previously processed message. Equation 1 represents the distribution of the messages-sequences at time t .

$$D(t) = \{E(N_i, N_j)(t)\} \quad (1)$$

where N_k is for node k and $E(a, b)$ is for the edge from node a to node b .

We consider that a MSG representing the w messages exchanged in a CAN bus at time t is *similar* to the MSG representing the w CAN messages exchanged during the following time slot $t + 1$ in the case of normal driving behavior and that injection of messages into the CAN bus disrupts this pattern [11]—see Figure 3 [11].

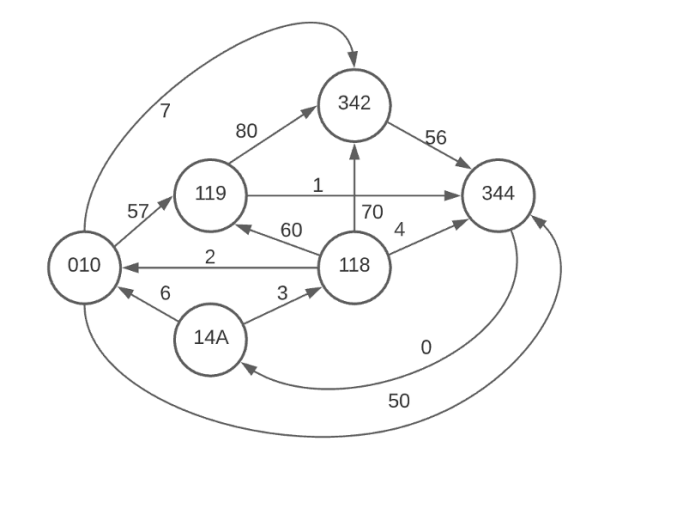
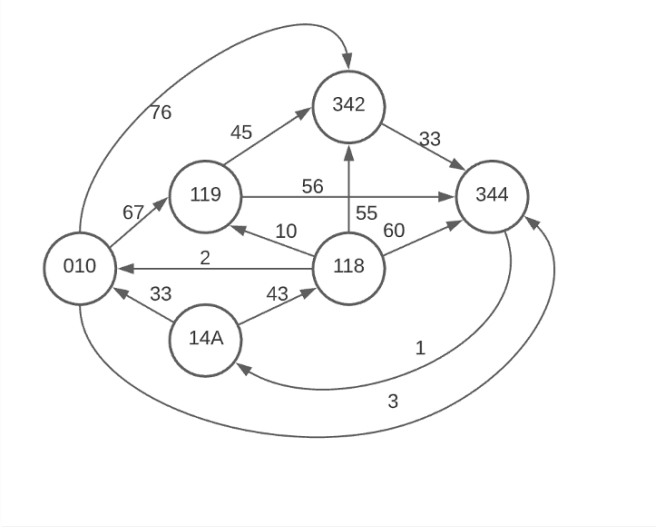


Fig. 3: Similarity of two messages-sequence graphs at successive time-window t (left) and $t + 1$ (right). The labels of the nodes are the CAN ID and the labels of the edges are number of times a message with the CAN ID source of the edge is followed by a message with the CAN ID destination of the same edge during the time-window. For example, 33 messages and 56 messages with ID 344 followed messages with ID 342 at resp. time-windows t and $t + 1$, which indicate a possible change of the behavior of the vehicle [11].

Equation 2 formulates the *Similarity* concept for the IDS. That is, the similarity Sim at time $t+1$ is the similarity of the distributions of the messages-sequences D at time t and at time $t+1$.

$$Sim(t + 1) = Similarity(D(t), D(t + 1)) \quad (2)$$

We use cosine similarity to measure the similarity between two MSGs of two successive time steps t and $t+1$. The metric measures the angle between two vectors, where the closer the value is to 1, the more similar the two vectors are. Then, we use *similarity threshold* technique to identify CAN message injections. The technique provides an accuracy of 97.32%—see Ref. [11] for details about the construction and accuracy of the technique.

We discuss in the following sections the architecture alternatives for monitoring the CAN Bus and using the similarity threshold technique to identify in real-time malicious injections of CAN messages.

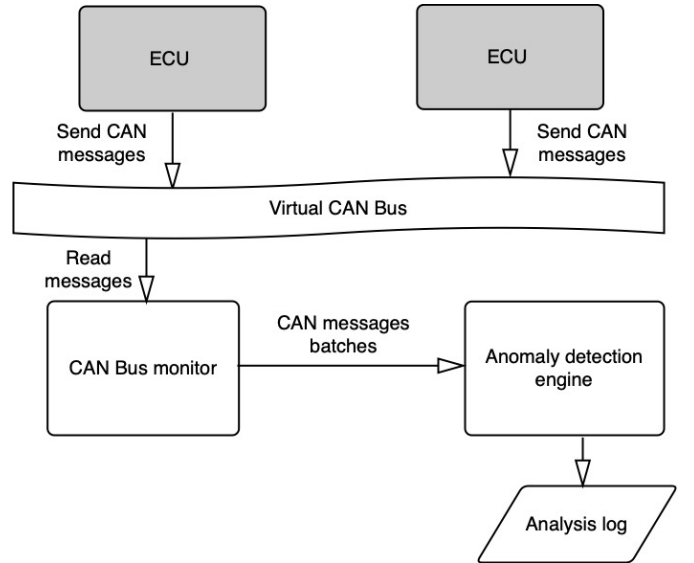


Fig. 4: High-level architecture of the system.

TABLE II: IDS architecture constraints.

Constraint	Value
Recommended maximum rate of injection of CAN messages	1908/sec
Rate of injection of CAN messages	1000/sec [10]
Reaction time constraint	2.5 sec
Detection speed of 1000 messages using the similarity threshold technique	0.094 seconds

- The CAN bus is designated for a maximum signaling of 1 Mbit/s/s [35] but 250kb/s is the recommended rate by the Society of Automotive Engineers (SAE) in J1939 standards [36], which transports up to 1908 CAN frame per second – $1908 = 250000 / (128 + 3)$ where the data payload is of 8 bytes.

- The break reaction time is less than 2.5 second for 90% of the drivers.[37]

- We postulate that the IDS needs to process the CAN messages batch file in less than 2.5 seconds, which is the upper bound of breaking reaction time [37].

IV. REAL-TIME ARCHITECTURE ALTERNATIVES OF IDS FOR CONNECTED VEHICLES

In a typical IDS environment, a set of ECUs inject CAN messages into the CAN bus, a CAN Bus monitor captures the messages exchanged in the bus, and the anomaly detection engine analyzes these messages to identify malicious message injections, as depicted in Figure 4. The *CAN bus monitor* reads the messages available in the CAN bus continuously and sends them to the anomaly detection engine in batches of, e.g., 1000 messages. The *anomaly detection engine* constructs a MSG from the received batch of messages, applies the cosine similarity threshold attack detection technique, and outputs the results of the evaluation.

The IDS must address three main requirements depicted in Table I. First, the response time of the IDS must be less than the expected breaking reaction time; satisfying this requirement makes the IDS a good candidate for e.g., a safety resilience mechanism that activates the breaks in case of detection of cyber-attacks. Second, the loss of CAN messages is not allowed, which is important for the reliability of IDS. Third, The IDS must run on a cheap ECU that has limited capabilities in terms of processor speed and memory size. In addition, Table II enumerates a set of constraints that the architecture shall satisfy. Since the goal of this paper is to assess architectures of real-time IDS, we use the existing anomaly detection module developed in previous work [11] and do not create new ones.

TABLE III: Practical concurrency scenarios of the IDS main components.

Architecture scenario	Concurrency technique of CAN Bus monitor	Concurrency technique of the anomaly detection agent	Use of a queue
Scenario 1 - The two components run in a single process	main process	main process	no
Scenario 2 -The two components run in a single process that includes one sub-process s	main process	child-process	no
Scenario 3 - The two components run in a single process with two threads	thread	thread	yes
Scenario 4 - The two components run in two processes	main process	main process	yes

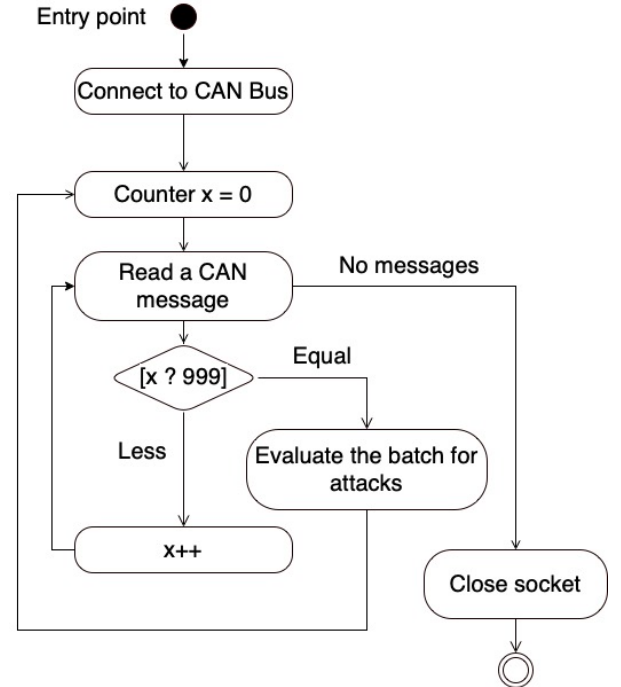


Fig. 5: IDS flowchart diagram.

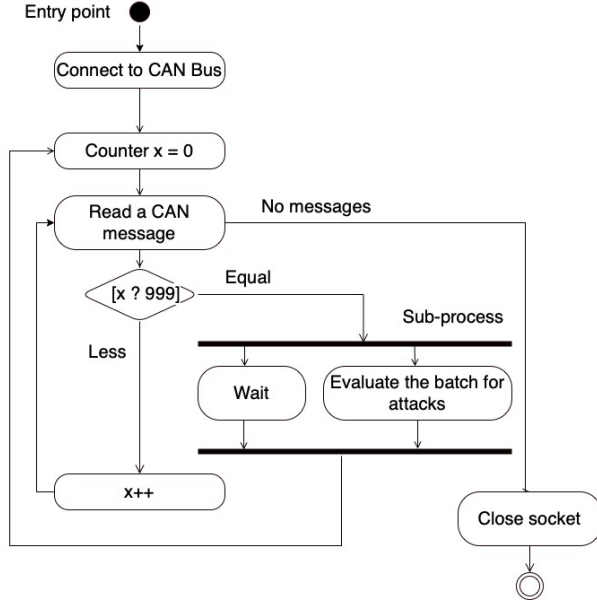


Fig. 6: Flowchart diagram of the IDS architecture of scenario 2.

To satisfy the first requirement, we implement the CAN Bus monitor and the anomaly detection engine using the C language. The anomaly detection engine uses PyObject library to call the data analysis module, passing it the CAN messages batches as a parameter, and parsing the analysis result.

Typically, each ECU services/reacts to each message it reads from the CAN bus (process or ignore) at a rate higher than the sending messages rate, to avoid loss of messages. In contrast, the IDS techniques process the CAN messages in batches, which takes much longer than one millisecond. Thus, the IDS would lose data if the CAN bus monitor and anomaly detection engine operate sequentially, as depicted in Figure 5, which violates the third requirements that we set for our real-time IDS: loss of CAN messages is not allowed. This problem could potentially be addressed by running the CAN Bus monitor and anomaly detection engine concurrently. The three concurrency techniques that could be used are: using separate processes, using sub-processes, and using threads. Table III shows the four potential architecture scenarios for using concurrency techniques of the CAN Bus monitor and the anomaly detection engine com-

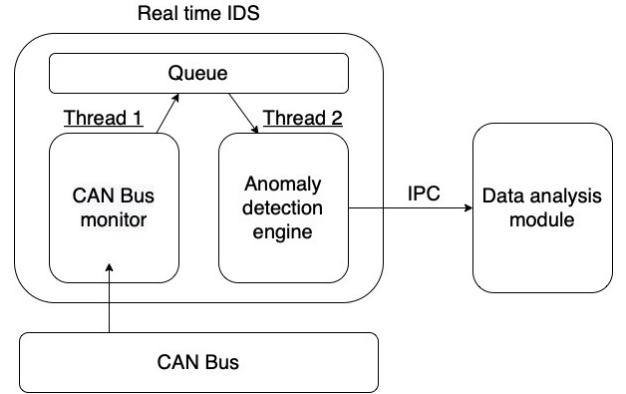


Fig. 7: Block diagram of the IDS architecture of scenario 3.

ponents to address the second and third requirements discussed above. We discuss in the following each of the architecture scenarios.

Scenario 1 - The two components run in a single process. In this scenario, the CAN Bus monitor and anomaly detection engine run sequentially as depicted by Figure 5. The CAN Bus monitor can potentially lose CAN messages that the ECUs send while the IDS busy analyzing the CAN messages batch it receives to identify potential attacks.

Scenario 2 - The two components run in a single process that includes one sub-process. The CAN bus monitor and the anomaly detection agent run in a single process. As depicted by Figure 6, the main process continuously reads the CAN messages from the CAN bus and creates a sub-process, that evaluates the batch for attacks, that is executed when there are enough messages for the batch. Note that the sub-processes become zombies at the end of their executions, and it is complicated to shut them down from the main processes.

Scenario 3 - The two components run in a single process with two threads. The CAN bus monitor and the anomaly detection agent run in a single process but in separate threads, as depicted by Figure 7. We use a queue to pass data between the two components to prevent losses of CAN messages. The anomaly detection engine uses Inter-Process Communication (IPC) technique to execute the data analysis module, which implements the technique discussed in Section III.

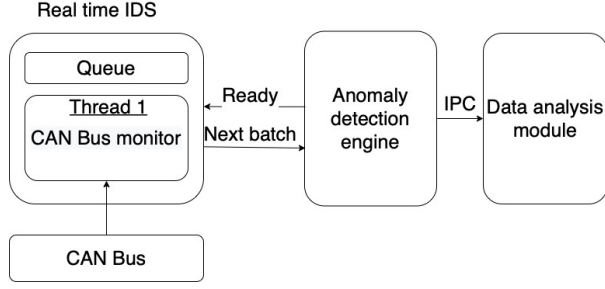


Fig. 8: Block diagram of the IDS architecture of scenario 4.

For completeness, there are two other variants of using threads with a single process besides the one discussed above, which are: (1) implementing the CAN bus monitor in a thread and the anomaly detection engine in the main process and (2) implementing the anomaly detection engine in a thread and the CAN bus monitor in the main process. We do not report the evaluation of these variants because they use the same concurrency technique and do not outperform the two threads variant.

Scenario 4 - The two components run in two processes. The CAN bus monitor and the anomaly detection agent run in separate independent processes, as depicted by Figure 8. We also use a queue to pass data between the two components to prevent losses of CAN messages.

V. EVALUATION OF THE FOUR REAL-TIME IDS ARCHITECTURE SCENARIOS

This section describes the evaluation environment and datasets, compares the anomaly evaluation times of the four architecture scenarios, and analyses the impact of CAN message injection speed on the message loss ratio of the four architecture scenarios.

A. Evaluation environment and datasets

We implemented the four architecture scenarios and deployed them to a Raspberry Pi that runs Raspbian 10, with four core processors of 1.2 MHz speed and 1GB of memory. Figure 9 shows the evaluation environment of the architecture scenarios. The environment uses an

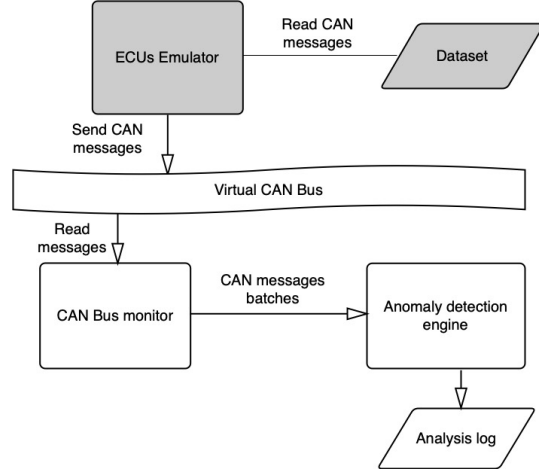


Fig. 9: Block diagram of the evaluation environment of the real-time IDS architecture scenarios.

TABLE IV: CAN messages datasets

No	Description	# of CAN messages
1	CAN Data for no injection of fabricated messages	23,963
2	CAN Data with injection of "FFF" as the speed reading	88,492
3	CAN Data with injection of "FFFF" as the RPM reading	30,308

TABLE V: Speed of simulating injection of CAN messages and evaluating them for attacks using architecture scenario 1.

	Normal messages	messages with injection of seed readings	messages with injection of RPM readings
Time to send 1000 CAN messages in seconds			
Min	0.946	0.925	1.015
Max	1.120	1.217	1.056
Average	0.992	1.008	1.023
Time to evaluate 1000 CAN messages in seconds			
Min	0.115	0.116	0.131
Max	0.271	0.274	0.180
Average	0.154	0.143	0.151

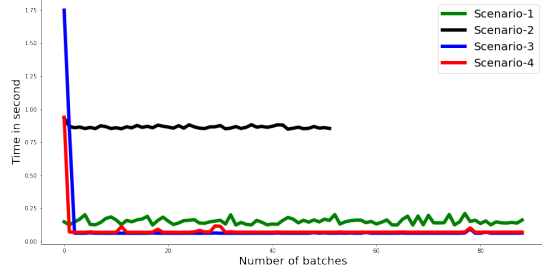


Fig. 10: Anomaly evaluation time of the four architecture scenarios.

ECUs emulator that simulates car ECUs, which sends messages periodically into Linux virtual CAN bus.

In this evaluation, we use datasets [38] of CAN bus messages for (1) normal driving behavior, (2) injection of fabricated speed reading messages onto the CAN bus, and (3) injection of fabricated RPM reading messages onto the CAN bus collected from an in-motion Ford Transit 500 2017 [10]. Table IV lists the datasets and the number of CAN messages in each of them.

The *ECUs emulator* reads the CAN messages stored in the dataset files and sends them through the virtual CAN bus. The messages are processed by the CAN bus monitor and anomaly detection engine in the four architecture scenarios. Table V provides the time that the ECUs emulator takes to send 1000 messages into the CAN Bus and the time that the IDS takes to evaluate 1000 CAN messages for the cases of normal messages dataset, messages with the injection of speed readings dataset, and messages with the injection of RPM readings dataset. We do not observe a big difference in processing a batch of normal CAN messages, messages with the injection of speed readings, and messages with the injection of RPM readings. We observe that the IDS takes an average 149 milliseconds to evaluate a batch of 1000 messages, while the time to send 1000 messages into the CAN bus is about 1,007 seconds. The IDS would lose about 149 CAN messages from each batch, which can impact the attack detection rate—i.e., ignoring 14,9% of the messages can impact the detection rate.

TABLE VI: Anomaly evaluation times and response times for the four architecture scenarios.

Architecture	Average time of sending 1000 CAN messages	Average evaluation time	Response time
Scenario 1 - Single process with no threads	998 ms	152 ms	1.15 sec.
Scenario 2 - Single process with one sub-process	944 ms	865 ms	1.809 sec
Scenario 3 - Single process with two threads	950 ms	90 ms	1.04 sec.
Scenario 4 - Two processes	945 ms	81 ms	1.026 sec.

B. Analysis of the anomaly evaluation time of the four architecture scenarios

We configured the ECU emulator to send the dataset "CAN Data with injection of "FFF" as the speed reading" with a rate of about 1000 messages through the virtual CAN Bus. Table VI shows the average time of sending a batch of CAN messages, the average evaluation time, and the average response time (time taken from collecting the first CAN message of the batch to output the result of the evaluation of the batch) of the four IDS architecture scenarios. We consider the *anomaly evaluation time* as the difference between the end of evaluating the messages batch for attacks and the end of reading 1000 CAN messages from the virtual CAN Bus. The data shows that anomaly evaluation time is way below the batch messages sending time. Figure 10 shows the anomaly evaluation time of the four selected real-time IDS architecture scenarios. The diagram shows that anomaly evaluation times of a real-time IDS designed as a single process, single process with two threads, and as two processes are below the brake reaction time (0.5 to 2.5 seconds), In addition, the anomaly evaluation times of the IDS designed as a single process with two threads and as two processes are too close.

We conclude that the IDS response time of a real-time IDS designed as a single process with no threads, as a single process with two threads, and as two processes is below the brake reaction time (assuming the rate of sending messages through the CAN Bus is about 1000 messages/second) which makes them a good candidate for real-time IDS for connected vehicles.

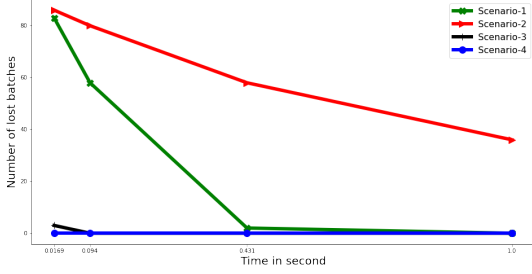


Fig. 11: Ratio of messages losses vs speed of sending 1000 CAN messages through the CAN bus in the four architecture scenarios.

C. Analysis of the reliability of the four architecture scenarios in terms of CAN message losses

Message loss ratio is the ratio of CAN messages that are sent through the CAN Bus by the ECUs emulator but are not received and processed by the anomaly detection engine. Theoretically, the CAN messages that the ECUs emulator sends through the CAN Bus while the single process IDS architecture is busy analyzing a batch of previously received messages are lost. Figure 11 shows the relationship between the messages loss ratio and speed of sending messages into the virtual CAN bus by the ECUs emulator. The figure shows that the ratio of messages loss decreases as the speed of sending CAN messages increases and it reaches 0% for the case of architecture scenario 1 and 2 and that there are no messages losses for the case of IDS designed as two processes—because the servicing time is smaller than the time to send the messages batch into the CAN Bus.

VI. CONCLUSION

Machine-learning-based IDSs techniques have been proposed for the detection of malicious injection of messages into the in-vehicle network of connected vehicles. Evaluations of such IDS have been performed offline, which limits the confidence in their use for real-time protection scenarios. We evaluated in this paper four architecture designs for real-time IDS for connected vehicles using an anomaly detection engine that uses similarities of graphs representing sequencing of CAN messages during a given period and a CAN dataset

collected from a moving vehicle under malicious speed reading message injections. The evaluation shows that a real-time IDS for a connected vehicle designed as two processes are reliable (no loss of messages) and have a low anomaly evaluation time that makes them a good candidate for real-time resilience mechanisms as a response to cyber-attacks.

VII. ACKNOWLEDGEMENT

The authors thank Arun Somani from Iowa State University for the thorough discussions about the research. This research is partly funded by Iowa State University's Regents Innovation Fund (RIF).

REFERENCES

- [1] R. Bosch GmbH, "Can specification v2.0." <http://esd.cs.ucr.edu/webres/can20.pdf>, 1991.
- [2] L. Ben Othmane, H. Weffers, M. Mohamad, and M. Wolf, *Wireless Sensor and Mobile Ad-Hoc Networks*, ch. A Survey of Security and Privacy in Connected Vehicles. Springer, 2015.
- [3] L. ben Othmane, H. Weffers, M. M. Mohamad, and M. Wolf, *Wireless Sensor and Mobile Ad-Hoc Networks: Vehicular and Space Applications*, ch. A Survey of Security and Privacy in Connected Vehicles, pp. 217–247. Springer, 2015.
- [4] Upstream Auto, "Upstream security's global automotive cybersecurity report." <https://www.upstream.auto/upstream-security-global-automotive-cybersecurity-report-2020/>, 2020. accessed on Feb 2020.
- [5] T. Hoppe, S. Kiltz, and J. Dittmann, "Security threats to automotive can networks – practical examples and selected short-term countermeasures," in *Computer Safety, Reliability, and Security* (M. D. Harrison and M.-A. Sujan, eds.), (Berlin, Heidelberg), pp. 235–248, Springer Berlin Heidelberg, 2008.
- [6] K. Koscher, A. Czeskis, F. Roesner, S. Patel, T. Kohno and S. Checkoway, D. McCoy, B. Kantor, D. Anderson, H. Shacham, and S. Savage, "Experimental security analysis of a modern automobile," in *Proc. 2010 IEEE Symposium on Security and Privacy (SP)*, (Los Alamitos, CA, USA), pp. 447–462, may 2010.
- [7] S. Checkoway, D. McCoy, B. Kantor, D. Anderson, H. Shacham, S. Savage, K. Koscher, A. Czeskis, F. Roesner, and T. Kohno, "Comprehensive experimental analyses of automotive attack surfaces," in *Proceedings of the 20th USENIX Conference on Security, SEC'11*, (USA), p. 6, USENIX Association, 2011.
- [8] W. Wu, R. Li, G. Xie, J. An, Y. Bai, J. Zhou, and K. Li, "A survey of intrusion detection for in-vehicle networks," *IEEE Transactions on Intelligent Transportation Systems*, vol. 21, no. 3, pp. 919–933, 2020.
- [9] C. Young, J. Zambreno, H. Olufowobi, and G. Bloom, "Survey of automotive controller area network intrusion detection systems," *IEEE Design Test*, vol. 36, no. 6, pp. 48–55, 2019.
- [10] L. Ben Othmane, L. Dhulipala, M. Abdelkhalek, N. Multari, and M. Govindarasu, "On the performance of detecting injection of fabricated messages into the can bus," *IEEE Transactions on Dependable and Secure Computing*, pp. 1–1, 2020.

- [11] M. Jedh, L. Ben Othmane, N. Ahmed, and B. Bhargava, "Detection of message injection attacks onto the can bus using similarities of successive messages-sequence graphs," *IEEE Transactions on Information Forensics and Security*, vol. 16, pp. 4133–4146, 2021.
- [12] G. Carel, R. Isshiki, T. Kusaka, Y. Nogami, and S. Araki, "Design of a message authentication protocol for can fd based on chaskey lightweight mac," in *2018 Sixth International Symposium on Computing and Networking Workshops (CANDARW)*, pp. 267–271, 2018.
- [13] H. J. Jo, J. H. Kim, H.-Y. Choi, W. Choi, D. H. Lee, and I. Lee, "Mauth-can: Masquerade-attack-proof authentication for in-vehicle networks," *IEEE Transactions on Vehicular Technology*, vol. 69, no. 2, pp. 2204–2218, 2020.
- [14] W. A. Farag, "Cantrack: Enhancing automotive can bus security using intuitive encryption algorithms," in *2017 7th International Conference on Modeling, Simulation, and Applied Optimization (ICMSAO)*, pp. 1–5, 2017.
- [15] C. Miller and C. Valasek, "Remote exploitation of an unaltered passenger vehicle." <https://www.youtube.com/watch?v=OobLb1Mcnrl>, 2015.
- [16] "Hacking tesla from wireless to can bus." <https://www.blackhat.com/docs/us-17/thursday/us-17-Nie-Free-Fall-Hacking-Tesla-From-Wireless-To-CAN-Bus-wp.pdf>. accessed on Dec. 2021.
- [17] "Experimental security assessment of bmw cars: A summary repor." https://keenlab.tencent.com/en/whitepapers/Experimental_Security_Assessment_of_BMW_Cars_by_KeenLab.pdf. accessed on Dec. 2021.
- [18] A. Greenberg, *Hackers Can Steal a Tesla Model S in Seconds by Cloning Its Key Fob*. wired., Sep 2018.
- [19] M.-J. Kang and J.-W. Kang, "Intrusion detection system using deep neural network for in-vehicle network security," *PLOS ONE*, vol. 11, pp. 1–17, 06 2016.
- [20] S.-F. Lokman, A. T. Othman, and M.-H. Abu-Bakar, "Intrusion detection system for automotive controller area network (can) bus system: a review," *EURASIP Journal on Wireless Communications and Networking*, vol. 184, no. 1, 2019.
- [21] S. Lokman, A. T. Bin Othman, and M. Abu-Bakar, "Optimised structure of convolutional neural networks for controller area network classification," in *2018 14th International Conference on Natural Computation, Fuzzy Systems and Knowledge Discovery (ICNC-FSKD)*, pp. 475–481, July 2018.
- [22] S. Longari, D. H. N. Valcarcel, M. Zago, M. Carminati, and S. Zanero, "Cannolo: An anomaly detection system based on lstm autoencoders for controller area network," *IEEE Transactions on Network and Service Management*, pp. 1–1, 2020.
- [23] S. Stachowski, R. Gaynier, and D. J. LeBlanc, "An assessment method for automotive intrusion detection system performance." <https://rosap.ntl.bts.gov/view/dot/41006>, April 2019.
- [24] K.-T. Cho and K. G. Shin, "Fingerprinting electronic control units for vehicle intrusion detection," in *Proc. of the 25th USENIX Conference on Security Symposium*, (Austin, TX, USA), pp. 911–927, 08 2016.
- [25] A. Taylor, S. Leblanc, and N. Japkowicz, "Anomaly detection in automobile control network data with long short-term memory networks," in *Proc. IEEE International Conference on Data Science and Advanced Analytics (DSAA)*, (Montreal, Canada), pp. 130–139, Oct 2016.
- [26] M. Levi, Y. Allouche, and A. Kontorovich, "Advanced analytics for connected car cybersecurity," in *Proc. IEEE 87th Vehicular Technology Conference (VTC Spring)*, (Porto, Portugal), pp. 1–7, 06 2018.
- [27] E. Seo, H. M. Song, and H. K. Kim, "Gids: Gan based intrusion detection system for in-vehicle network," in *Proc. 2018 16th Annual Conference on Privacy, Security and Trust (PST)*, pp. 1–6, Aug 2018.
- [28] G. D'Angelo, A. Castiglione, and F. Palmieri, "A cluster-based multidimensional approach for detecting attacks on connected vehicles," *IEEE Internet of Things Journal*, pp. 1–1, 2020.
- [29] R. Islam, R. U. D. Refat, S. M. Yerram, and H. Malik, "Graph-based intrusion detection system for controller area networks," *IEEE Transactions on Intelligent Transportation Systems*, pp. 1–10, Oct, 2020.
- [30] D. S. Cruzes and L. ben Othmane, *Empirical Research for Software Security: Foundations and Experience*, ch. Threats to Validity in Software Security Empirical Research, pp. 275–300. Taylor & Francis Group, LLC, 2017.
- [31] C. Valasek and C. Miller, *A Survey of Remote Automotive Attack Surfaces*. Black Hat USA 2014, Aug 2014.
- [32] T. Matsumoto, M. Hata, M. Tanabe, K. Yoshioka, and K. Oishi, "A method of preventing unauthorized data transmission in controller area network," in *2012 IEEE 75th Vehicular Technology Conference (VTC Spring)*, pp. 1–5, 2012.
- [33] S. Boddupalli and S. Ray, "Redem: Real-time detection and mitigation of communication attacks in connected autonomous vehicle applications," *IFIP advances in information and communication technology*, vol. 574, pp. 105–122.
- [34] S. Boddupalli, A. S. Rao, and S. Ray, "Resilient cooperative adaptive cruise control for autonomous vehicles using machine learning," 2021.
- [35] Texas instrument, "Introduction to the controller area network (can)." <https://www.ti.com/lit/an/sloa101b/sloa101b.pdf>. Application Report SLOA101B–August 2002–Revised May 2016.
- [36] Society of Automotive Engineers, "J1939 recommended practice for a serial control and communications vehicle network." https://www.sae.org/publications/collections/content/j1939_dl/.
- [37] "Transportation engineering online lab manual - brake reaction time." https://www.webpages.uidaho.edu/niatt_labmanual/chapters/geometricdesign/theoryandconcepts/BrakeReactionTime.htm. accessed on Dec 2021.
- [38] L. Ben othmane and L. Dhulipala, "Injection of rpm and speed reading messages onto the can bus of a moving vehicle." <https://dx.doi.org/10.21227/sl1jy-h433>, 2020. IEEE Dataport.